

2026-05-05-p1-f11-session-transcript-resume-design

Phase 1 / Feature 11 — Session Transcript Resume

Date: 2026-05-05 **Status:** Approved (auto-approved per programme cadence) **Programme:** CLI-Agent Fusion — Phase 1 port from claude-code

1. Goal

Persist session transcripts to disk and resume them via `helixcode --resume` (most recent in current project) or `helixcode --continue` (most recent across all projects). Transcripts append after every user/assistant message so an unexpected crash doesn't lose context. `/sessions slash command + helixcode sessions cobra subcommand expose list/show <id>/resume <id>/delete <id>`. Identity scoping computes the Git root for the session's project; cwd fallback when not in a repo. JSONL transcripts + JSON metadata in `~/.local/share/helixcode/sessions/<id>/`.

2. Architecture

New `internal/session/resume.go` holds `ResumeFinder`, `ResumeMode` enum (`ResumeProject / ResumeGlobal`), `SessionMetadata` struct. Persistence via JSONL files in `$XDG_DATA_HOME/helixcode/sessions/<id>/transcript.jsonl` plus `metadata.json`. New `internal/session/transcript_store.go` implements the `SessionStore` interface using `stdlib os/encoding/json` only — no DB. Existing `internal/session/session_manager.go` extended with `Append(msg Message) error` (called after each turn) and `Resume(ctx, id) (*Session, error)`. CLI flags `--resume` and `--continue` parsed before existing dispatcher; resolved session ID injected into the session manager startup. `/sessions slash + helixcode sessions cobra mirror F09/F10` patterns. Identity computed via `git rev-parse --show-toplevel`; on error, falls back to `cwd`.

3. Components

3.1 New files

- `helix_code/internal/session/resume.go` — `ResumeFinder`, `ResumeMode`, `SessionMetadata`, `FindResumeTarget`, `ResumeSession`
- `helix_code/internal/session/resume_test.go`
- `helix_code/internal/session/transcript_store.go` — JSONL transcript I/O (`Append`, `Read`, `List`, `Delete`)
- `helix_code/internal/session/transcript_store_test.go`
- `helix_code/internal/session/identity.go` — `computeProjectIdentity()` (Git root, fallback `cwd`)

- `helix_code/internal/session/identity_test.go`
- `helix_code/internal/commands/sessions_command.go` — `/sessions slash`
- `helix_code/internal/commands/sessions_command_test.go`
- `helix_code/cmd/cli/sessions_cmd.go` — cobra subcommand
- `helix_code/cmd/cli/sessions_cmd_test.go`
- `helix_code/tests/integration/sessions_resume_test.go` — `// go:build integration`
- `challenges/p1-f11-session-resume/CHALLENGE.md` + `run.sh`

3.2 Modified

- `helix_code/internal/session/session_manager.go` — `Append(msg Message) error` (calls `store`), `Resume(ctx, id) error`, `ExposeCurrentID()`.
- `helix_code/cmd/cli/main.go` — parse `--resume/--continue` flags before dispatcher; if set, call `ResumeFinder.FindResumeTarget + sessionMgr.Resume.Register /sessions slash + helixcode sessions cobra`.

3.3 Types

```

type ResumeMode string
const (
    ResumeProject ResumeMode = "project" // current project only
    ResumeGlobal   ResumeMode = "global"  // across all projects
)

type SessionMetadata struct {
    SessionID      string    `json:"session_id"`
    ProjectPath    string    `json:"project_path"` // git toplevel
                    or cwd
    ProjectName    string    `json:"project_name"` // basename of
                    ProjectPath
    StartedAt      time.Time `json:"started_at"`
    LastActivity   time.Time `json:"last_activity"`
    MessageCount   int       `json:"message_count"`
    IsActive       bool      `json:"is_active"`
    BranchName     string    `json:"branch_name,omitempty"`
}

type SessionStore interface {
    ListSessionMetadata(ctx context.Context, projectPath string)
        ([]SessionMetadata, error)
    GetSessionMetadata(ctx context.Context, sessionID string)
        (*SessionMetadata, error)
    UpdateSessionMetadata(ctx context.Context, meta
        SessionMetadata) error
    Append(ctx context.Context, sessionID string, msg Message)
        error

```

```

    ReadTranscript(ctx context.Context, sessionID string)
        ([]Message, error)
    DeleteSession(ctx context.Context, sessionID string) error
}

type ResumeFinder struct {
    store SessionStore
}

func NewResumeFinder(store SessionStore) *ResumeFinder
func (rf *ResumeFinder) FindResumeTarget(ctx context.Context,
    mode ResumeMode, currentProject string)
    (*SessionMetadata, error)
func (rf *ResumeFinder) Resume(ctx context.Context, sessionID
    string) ([]Message, *SessionMetadata, error)

```

3.4 Storage layout

```

$XDG_DATA_HOME/helixcode/sessions/
├── <session-id>/
│   ├── metadata.json      # SessionMetadata
│   └── transcript.jsonl    # one JSON-encoded Message per line

```

Per-message append: `os.OpenFile(path, O_APPEND|O_CREATE|O_WRONLY, 0644)` then write `json.Marshal(msg) + "\n"`. After each append, update `metadata.json`'s `LastActivity` and `MessageCount`. Crash safety: even partial writes leave the file readable up to the last newline.

3.5 Identity

```

// computeProjectIdentity returns the Git toplevel for the cwd,
//   or the cwd
// itself when not in a repo. Used to scope ResumeProject
//   lookups.
func computeProjectIdentity() (string, error)

```

Implementation: `exec.Command("git", "rev-parse", "--show-toplevel");` on error or non-zero exit, return `os.Getwd()`. Errors surfaced only on `os.Getwd()` failure.

3.6 User surfaces

CLI flags (top-level): `- helixcode --resume` — resume most recent session in current project `- helixcode --continue` — resume most recent session across all projects `- helixcode --resume <id>` — resume specific session by ID

`/sessions slash: - /sessions` (default list) — table NAME / ID / STARTED / LAST-ACTIVITY / MSG-COUNT `- /sessions show <id>` — metadata + last 20 messages `- /sessions resume <id>` — switch the running CLI session to that transcript `- /sessions delete <id>` — remove from disk

```
helixcode sessions cobra: -helixcode sessions list [--all] (--all for
global; default project-scoped) -helixcode sessions show <id> -helixcode
sessions delete <id>
```

4. Data flow

4.1 Startup with --resume

```
helixcode --resume
├─ project := computeProjectIdentity()
├─ finder := NewResumeFinder(store)
├─ meta, err := finder.FindResumeTarget(ctx, ResumeProject, project)
├─ if err: print "no sessions to resume"; start fresh session
├─ messages := finder.Resume(ctx, meta.SessionID)
├─ sessionMgr.LoadFromTranscript(meta.SessionID, messages)
└─ proceed with normal CLI run; appends continue on the existing transcript
```

4.2 Per-turn append

```
agent receives user message
├─ sessionMgr.Append(userMsg) // store.Append + update metadata
├─ sessionMgr.Append(assistantMsg)
└─ ...
```

4.3 /sessions list

```
SessionsCommand.list()
├─ project := computeProjectIdentity()
├─ metas := store.ListSessionMetadata(ctx, project) // project-scoped
├─ render tabwriter table
└─ return
```

/sessions list --all (variant) lists across all projects.

5. Error handling

- **Disk write failure:** log WARN, continue (don't crash on transcript write failure — agent stays responsive).
- **Corrupted JSONL line:** skip the malformed line, log path + line number; subsequent lines still parsed.
- **Missing metadata.json:** synthesize from JSONL (read first message timestamp = StartedAt; last message timestamp = LastActivity).
- **Concurrent append:** file lock via O_APPEND is atomic at the syscall level for line-sized writes; no explicit lock.
- **--resume with no sessions:** graceful — print “no sessions to resume”, start fresh.

Anti-bluff (CONST-035 / §11.9)

- Challenge spawns a fresh session, appends 3 messages, kills the session, restarts with --resume, asserts all 3 messages are visible. Real subprocess, real disk I/O.
- Tests use real tempdirs + real JSONL appends — no mocks for the transcript store.

- Anti-bluff smoke clean across all new files.

6. Testing

Unit tests: - `TestComputeProjectIdentity_GitRoot` (real `git init` in `tempdir`) -
`TestComputeProjectIdentity_NoGitFallback` (`cwd` fallback) -
`TestTranscriptStore_AppendReadRoundTrip` -
`TestTranscriptStore_HandlesCorruptedLine` -
`TestTranscriptStore_MetadataResynth` (delete `metadata.json`, re-derive) -
`TestTranscriptStore_DeleteSession` (`rm -rf` the session dir) -
`TestResumeFinder_FindMostRecentInProject` -
`TestResumeFinder_FindMostRecentGlobal` -
`TestResumeFinder_NoSessions` -
`TestSessionManager_AppendUpdatesMetadata` - `TestSlashSessions_List/`
`Show/Resume/Delete/Default/UnknownErrors` - `TestSessionsCmd_List/`
`Show/Delete`

Integration test (real fs + real append cycle): -
`TestSessionsResume_PersistsAcrossProcessRestart` -
`TestSessionsResume_GlobalFindsMostRecentAcrossProjects`

Challenge: real subprocess writes 3 messages, restarts with `--resume`, asserts transcript fully recovered.

7. Cross-platform

`os.UserConfigDir()` / `os.UserHomeDir()` work on all platforms. `XDG_DATA_HOME` falls back to `$HOME/.local/share` on Linux, `~/Library/Application Support` on macOS, `%LOCALAPPDATA%` on Windows. Cross-compile linux is the canonical check.

8. Out of scope (deferred)

- Multi-session merging / branching
- Cloud sync (sessions are local-only)
- Per-message search across transcripts
- PR URL + branch linkage (porting doc mentions; deferred to F11.5)
- Encrypted-at-rest transcripts

9. Constitutional compliance

- §11.9: Challenge captures real cross-process resumption.
- CONST-042: transcripts may contain secrets (e.g., paste of an API key in conversation). Mode 0644 chosen for ergonomic editing; future hardening to 0600 if surfaced as a concern.
- CONST-043: non-force pushes.

10. Open questions resolved

Q	Answer
Q1: where does resume logic live	(A) <code>internal/session/resume.go</code>
Q2: storage format	(A) JSONL on disk + JSON metadata
Q3: user surface	(C) Both <code>--resume/--continue</code> flags AND <code>/sessions slash + helixcode sessions cobra</code>
Q4: persistence cadence	(A) Append-after-each-turn
Q5: identity scoping	(C) Git root with cwd fallback